Hyperparameters and Their Effects

How Training Configuration Shapes LLM Performance

LECTURE 7 | DURATION · 60 MIN | CONFIGURATION → OPTIMIZATION

TOPICS COVERED

- Learning Rate
- Batch Size
- Epochs
- Gradient Accumulation
- AdamW
- Warmup & Schedules
- Weight Decay
- Dropout
- Gradient Clipping
- Sequence Length
- LoRA Rank & Alpha
- Temperature
- Top-k / Top-p



WHERE ARE WE IN THE COURSE?

From What Each Knob Does to Tuning Them

LECTURE 4
PEFT → LoRA → QLoRA

→

LECTURE 5
Inference + Latency

→

LECTURE 6
Compression

→

LECTURE 7
Hyperparameters and Their Effects

YOU ARE HERE

→

LECTURE 8
Hyperparameter Optimization + Complete Production Case Study

> **Lecture 7:** What does each hyperparameter do?
>
> **Lecture 8: How do we find good values systematically?**

Training fundamentals → Core training hyperparameters → LoRA & generation knobs → Diagnosis + experiment discipline

SECTION 01 / 04

Fundamentals

Parameters are learned. Hyperparameters are chosen. Confusing them confuses everything.

01 Parameters vs Hyperparameters

02 Who Changes What?

03 Effects Beyond Accuracy

01

SECTION 01 · THE DISTINCTION

Parameters vs Hyperparameters

MODEL PARAMETERS

values **learned during training**

- Weights
- Biases
- LoRA A / B matrix values

they change through gradient descent

HYPERPARAMETERS

values **chosen by us** before or during the training configuration

- Learning Rate
- Batch Size
- Epochs
- Sequence Length
- Weight Decay
- LoRA Rank
- LoRA Alpha
- Dropout

`Learning Rate = 2e-4` — chosen by us

`Weight = 0.73684` — learned by the model

PARAMETERS = what the student learns · **HYPERPARAMETERS** = how we organize the student's learning

→ **Hyperparameters control the learning process; parameters are the result of that learning process.**

SECTION 01 · CAUSE AND SCOPE

Who Changes What? — and Effects Beyond Accuracy

THE CAUSAL CHAIN

Training Data

↓

Hyperparameter Configuration

↓

Training Algorithm

↓

Updates

↓

Model Parameters

hyperparameters shape the process — the algorithm changes the parameters

Bad hyperparameters can affect:

MODEL QUALITY

accuracy · F1 · perplexity · generalization

TRAINING

convergence · stability · overfitting · underfitting

RESOURCES

GPU memory · training time · compute cost

DEPLOYMENT (indirect)

latency · output length · memory · response quality

SECTION 02 / 04

Learning Rate

The size of each optimization step — often the most influential knob of all.

01 The Update Rule

02 Too Small / Too Large

03 Finding a Suitable Value

02

SECTION 02 · DEFINITION

What Is the Learning Rate?

$$\theta_{new} = \theta_{old} - \eta \nabla L$$

η = learning rate

∇L = gradient of the loss

The learning rate controls the size of each optimization step.

VERY SMALL STEPS



safe but slow

VERY LARGE STEPS



may overshoot

SUITABLE STEPS



stable progress toward the minimum

> “The learning rate is often one of the most influential training hyperparameters.”

SECTION 02 · TWO FAILURE MODES

Learning Rate Too Small — or Too Large?

TOO SMALL

- loss decreases very slowly
- training takes many steps
- model may appear not to learn
- compute is wasted

LOSS EXAMPLE

```
Epoch 1 loss = 2.40
Epoch 2 loss = 2.37
Epoch 3 loss = 2.35
```

updates may be too conservative

slow loss reduction does not automatically prove the LR is too small — **diagnosis requires multiple signals.**

TOO LARGE

- loss jumps up and down
- loss may explode
- training becomes unstable
- model quality deteriorates
- NaNs may occur in severe cases

STEP LOSSES

```
Step 1 = 2.1
Step 2 = 1.8
Step 3 = 4.9
Step 4 = 8.2
```

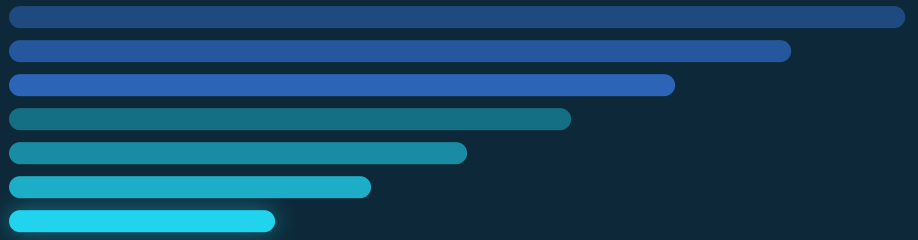
```
loss \/ \  /\__/ —
```

oscillating / exploding

SECTION 02 · WHAT GOOD LOOKS LIKE

A Suitable Learning Rate — and Why It Depends

SUITABLE



healthy loss curve

we usually want:

- stable progress
- meaningful loss reduction
- good validation performance

not necessarily perfectly smooth training loss.

DEPENDS ON TRAINING METHOD

do not teach one universal learning rate — **different configurations may need different values:**

- Full Fine-Tuning
- Selective Fine-Tuning
- LoRA
- QLoRA

LoRA adapters can often tolerate learning rates different from full-model updates

There is no single universally correct learning rate for all LLMs.

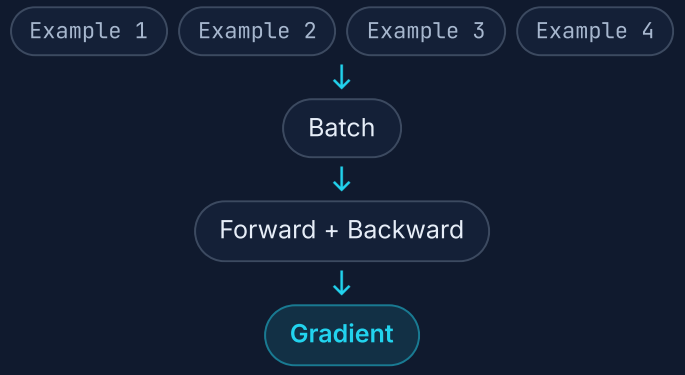
SECTION 03 · BATCH SIZE

What Is Batch Size? — and Small vs Large

CONCEPT

Suppose dataset has 1,000 examples, batch size = 4

```
1000 / 4 = 250 batches per epoch
```



SMALLER BATCH

BENEFITS
+ lower GPU memory
+ more frequent updates
+ gradient noise can sometimes aid generalization

COSTS
– lower hardware utilization
– noisier gradient estimates

LARGER BATCH

BENEFITS
+ improved hardware utilization
+ smoother gradient estimates
+ potentially higher throughput

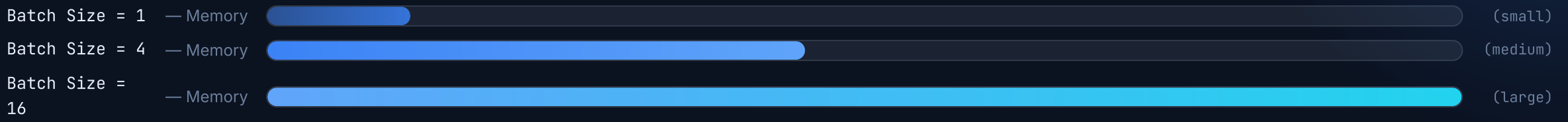
COSTS
– more GPU memory
– may require learning-rate adjustment

! Larger batch is not automatically better.

SECTION 03 · MEMORY & ACCUMULATION

GPU Memory — and Gradient Accumulation

GPU MEMORY BY BATCH SIZE



actual scaling depends on: `sequence length` `model` `precision` `architecture` `activation memory` `implementation`

Batch size is often limited by GPU memory before it is limited by theory.

GRADIENT ACCUMULATION

GPU fits micro-batch = 2 · we want effective batch ≈ 8 · use accumulation steps = 4

Batch 1	→	Batch 2	→	Batch 3	→	Batch 4	→	Optimizer Step
gradients stored		accumulate		accumulate		accumulate		update θ once

Effective Batch Size = **Per-Device Batch × Gradient Accumulation Steps × Number of Devices**

2 × 4 × 2 = 16 effective batch size

SECTION 03 · CLARIFICATIONS

One Clarification — then Epochs & Steps

Gradient accumulation **does not magically provide the same runtime behavior** as physically processing the full effective batch simultaneously in every respect.

- extremely useful when: desired batch > GPU memory capacity
- main benefit: larger effective batch without storing the entire batch at once

EPOCH

one epoch = one complete pass through the training dataset

```
Dataset = 10,000 examples
Epoch 1 → all 10,000
Epoch 2 → all 10,000 again
Epoch 3 → again
```

STEPS VS EPOCHS

```
10,000 examples · batch size = 10 → ≈ 1,000 steps / epoch
for 3 epochs → ≈ 3,000 optimization iterations
```

with gradient accumulation, distinguish: `micro-batch steps` vs **`optimizer steps`** — *these terms can differ in logs and frameworks.*

SECTION 03 · HOW MANY PASSES

Too Few Epochs vs Too Many Epochs

TOO FEW → UNDERFITTING

- training loss remains high
- validation performance poor
- task behavior not learned sufficiently

training stopped before useful adaptation completed

TOO MANY → OVERFITTING

Training loss	↓ ↓ ↓ ↓ ↓
Validation loss	↓ ↓ then ↑ ↑

MODEL MAY MEMORIZE: exact wording · noise · idiosyncrasies · rare artifacts

! More epochs ≠ automatically more learning quality.

SECTION 04 / 04

Optimizer, Warmup & Regularization

How the learning signal becomes updates — and how we keep training **stable** and **honest**.

AGENDA

01 AdamW

02 Weight Decay & Dropout

03 Warmup & Schedules

04 Gradient Clipping

SECTION 04 · OPTIMIZER

What Does an Optimizer Do? — AdamW

↗ GRADIENT

tells us: which direction should the parameter move?

OPTIMIZER

decides: how to use gradients to update parameters

BASIC RULE

$$\theta_{t+1} = \theta_t - \eta \nabla L$$

basic gradient descent — modern optimizers use more information.

Σ ADAM FAMILY

LLM training commonly uses optimizers from the Adam family. Conceptually Adam tracks:

1. gradient information
2. running average of gradients
3. running average of squared gradients

this provides adaptive parameter updates.

ADAMW for many Transformer training pipelines, AdamW is common — it handles **weight decay separately** from the adaptive gradient update.

Gradient → **AdamW** → Adaptive update → Parameter changes

no need to derive Adam mathematically in this course.

The optimizer defines how the learning signal becomes parameter updates.

SECTION 04 · REGULARIZATION

Weight Decay & Dropout

WEIGHT DECAY

a regularization technique that **discourages parameters from growing excessively large**

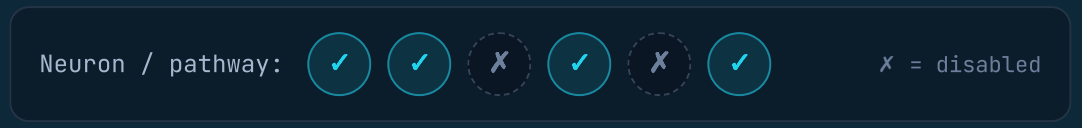
`Task Loss + Regularization pressure`

- can improve generalization and reduce overfitting in some settings
- `too little` → overfitting risk ↑
- `too much` → useful parameter learning suppressed, underfitting risk ↑

Regularization must also be balanced.

DROPOUT

during training, dropout **temporarily disables a fraction of activations / contributions**



PURPOSE: regularization | potentially reduces overfitting

⚠ too much dropout → learning becomes difficult, useful model capacity reduced during training.

SECTION 04 · STABILITY TOOLS

Warmup, LR Schedule & Gradient Clipping

WARMUP

starting immediately with a large learning rate can destabilize early training



learning rate gradually increases from a small value

let training enter a stable regime before using the full planned learning rate.

LR SCHEDULE

`Warmup` → `Peak LR` → `Gradual Decay`

`linear decay` `cosine decay`

`constant after warmup`

main idea: **learning rate can change during training**

we do not optimize schedulers yet.

GRADIENT CLIPPING

sometimes gradients become very large

`normal gradient` → use normally

`huge gradient` → clip to maximum norm / value

reduce the risk of unstable updates caused by exploding gradients

`max_grad_norm = 1.0` is common in many pipelines — but not a universal constant.

SEQUENCE LENGTH · MEMORY & COMPUTE

Maximum Sequence Length — and Why It Is Expensive

FORMAT

controls how many tokens each training example can contain after formatting / tokenization, subject to model limits:

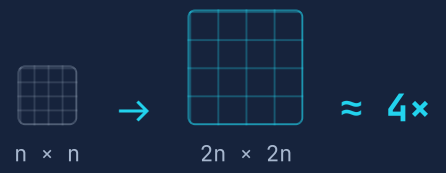
`max_seq_length = 512` 1024 2048 4096

`max_seq_length ↑` → More Tokens → More Activations → More Memory → More Compute

FULL SELF-ATTENTION SCALING

for standard full self-attention, the attention-score interaction count scales roughly as $O(n^2)$ with sequence length n

if sequence length doubles: n → 2n, the attention-score matrix grows from n × n to 2n × 2n — roughly 4× as many pairwise positions



actual end-to-end training cost depends on the architecture and implementation.

SEQUENCE LENGTH → LORA CONFIGURATION

The Length Trade-Off — and Back to LoRA

TRADE-OFF

TOO SHORT risk:

- important content truncated
- incomplete examples
- poor task learning

TOO LONG risk:

- higher GPU memory
- slower training
- lower batch size
- higher cost

Choose enough context for the task — without wasting capacity.

LORA HYPERPARAMETERS RECALL

from Lecture 4

$$\Delta W = (\alpha / r) \cdot B\,A$$

important configuration choices:

- Rank r
- Alpha α
- LoRA Dropout
- Target Modules
- Learning Rate

→ next: **what each LoRA knob does.**

LORA CONFIGURATION · CAPACITY & SCALE

LoRA Rank r & Alpha α

RANK r

r = 4 | r = 8 | r = 16 | r = 32

INCREASING R:

- Adaptation Capacity ↑
- Trainable Parameters ↑
- Memory Requirement ↑

⚠ Higher rank ≠ guaranteed higher quality.

ALPHA α

$$(\alpha / r) \cdot B A$$

alpha controls the scaling of the LoRA update in the common formulation

- ↘ small α → smaller LoRA contribution
- ↗ larger α → stronger scaled contribution

Do not treat $\alpha = 2r$ as a law — it is merely one configuration pattern sometimes used.

LORA CONFIGURATION · SCOPE & PRACTICE

LoRA Dropout, Target Modules — and Your Lab

LORA DROPOUT

regularizes the LoRA adaptation path

same idea as dropout — applied to the adaptation path

TARGET MODULES

could include architecture-dependent modules such as:

q proj | k proj | v proj | o proj | + sometimes MLP projections

→ more target modules → Adaptation flexibility ↑

↑ but also Trainable parameters ↑ · Training cost ↑

YOUR LABORATORY CONNECTION

Model: Qwen/Qwen2.5-0.5B-Instruct

Epochs: 5

Learning Rate: 2e-4

Trainable Parameters: ≈ 1.08 million

Total Parameters: ≈ 495 million

"Why did changing the dataset and number of epochs affect performance?"

think about interactions, not isolated numbers.

INTERACTIONS · TWO FAMILIES

Hyperparameters Interact — and Split in Two Families

! THIS IS VERY IMPORTANT

do not think these are independent:

Learning Rate | Batch Size | Epochs | Sequence Length

Larger Batch → may motivate different LR

Longer Sequence → higher memory → may force smaller Batch

More Epochs → higher overfitting risk → may require stronger regularization

> *"Hyperparameters form a system, not a collection of independent knobs."*

TRAINING HYPERPARAMETERS

Learning Rate | Batch Size | Epochs | Weight Decay | Warmup | Sequence Length | LoRA Rank

GENERATION HYPERPARAMETERS

Temperature | Top-k | Top-p | Max New Tokens | Repetition Penalty

training hyperparameters affect **how the model learns** — generation hyperparameters affect **how a trained model produces responses**.

GENERATION HYPERPARAMETERS · SAMPLING

Temperature — and Top-k / Top-p

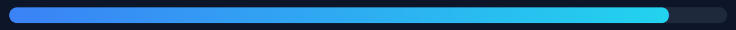
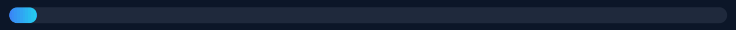
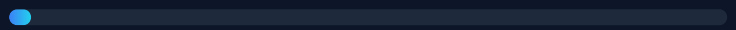
TEMPERATURE

modifies the **sharpness** of the next-token probability distribution

↓ lower temperature → more deterministic / concentrated

↑ higher temperature → more diverse / random

AT LOW TEMPERATURE

Delhi		0.92
Mumbai		0.04
Kolkata		0.03

at higher temperature the distribution becomes flatter

Temperature does not give the model new knowledge — it changes sampling behavior.

TOP-k / TOP-P

TOP-K — only sample from the k highest-probability tokens

$top_k = 5$

TOP-P / NUCLEUS SAMPLING — choose the smallest candidate set whose cumulative probability reaches approximately p

$top_p = 0.9$

control generation diversity.

GENERATION → DIAGNOSIS

Max New Tokens — and the Cause-and-Effect Table

MAX NEW TOKENS `max_new_tokens = 50` vs `max_new_tokens = 1000`

AFFECTS: maximum response length | worst-case decode duration | serving cost | memory / runtime

Lecture 5 connection: output length is also a latency decision.

HYPERPARAMETER	↓ TOO LOW	↑ TOO HIGH
Learning Rate	slow learning	unstable training
Batch Size	noisy / low utilization	high memory
Epochs	underfitting	overfitting
Weight Decay	weak regularization	underfitting
Dropout	weak regularization	learning reduced
Sequence Length	truncation	high memory / compute
LoRA Rank	insufficient capacity	unnecessary parameters / cost
Temperature	deterministic	overly random
Max New Tokens	truncated answer	unnecessary latency / cost

these are typical tendencies — not guaranteed outcomes.

DIAGNOSIS · CLASSROOM ACTIVITY

Diagnose the Symptom — Then the Scenario

FOUR TRAINING SYMPTOMS

SYMPTOM 1 **Training loss barely changes**

causes: LR too small · insufficient capacity · poor data · frozen / incorrect target parameters · implementation error

SYMPTOM 2 **Loss becomes unstable**

causes: LR too large · numerical instability · bad batch / example · exploding gradients

SYMPTOM 3 **Training improves, validation worsens**

likely: **overfitting** → response: fewer epochs · stronger regularization · more / better data

SYMPTOM 4 **CUDA out of memory**

levers: batch size ↓ · sequence length ↓ · gradient accumulation ↑ · memory-efficient training · QLoRA · checkpointing where appropriate

DIAGNOSE THE HYPERPARAMETER

A **Training loss oscillates wildly. First hyperparameter to investigate?**

→ Learning rate.

B **GPU out of memory after increasing sequence length. Why?**

→ More sequence-related activation / attention memory.

C **Training accuracy 99%, validation 76%. Likely?**

→ Overfitting — investigate epochs, regularization, dataset.

D **Model outputs highly random answers. Inference setting to inspect?**

→ Temperature / sampling configuration.

CLASSROOM CHECK · TRUE OR FALSE

Common Misconceptions — and How to Experiment

TRUE / FALSE

8 STATEMENTS

Hyperparameters are automatically learned by gradient descent.	FALSE
Higher learning rate always trains faster.	FALSE
More epochs always improve validation performance.	FALSE
Larger batch size always improves accuracy.	FALSE
Gradient accumulation can help emulate a larger effective batch.	TRUE
Larger LoRA rank always produces a better model.	FALSE
Longer sequence length has no memory consequence.	FALSE
Generation temperature changes the model's learned weights.	FALSE

ONE HYPERPARAMETER AT A TIME

Change everything → Performance improves → Why? ???

instead, experiments need **traceability**. Record:

- Run ID
- Learning Rate
- Batch Size
- Epochs
- Rank
- Alpha
- Sequence Length
- Validation Metric
- Runtime
- Memory

this is the bridge to systematic optimization.

EXPERIMENT DISCIPLINE · EVALUATE HONESTLY

An Experiment Table — and Where to Evaluate

EXPERIMENT TABLE

RUNS A–E

RUN	LR	BATCH	EPOCHS	RANK	VAL F1
A	1e-4	4	3	8	0.84
B	2e-4	4	3	8	0.89
C	5e-4	4	3	8	0.79
★ D	2e-4	4	5	8	0.91
E	2e-4	4	8	8	0.88

"Which run is best?" → based only on validation F1: **Run D**

"What about training time, memory and latency?" — **this prepares Lecture 8.**

WHERE TO EVALUATE

TRAINING SET → learn parameters

↓

VALIDATION SET → choose hyperparameters

↓

TEST SET → final unbiased evaluation

Hyperparameter tuning on the test set causes **test leakage**.

connect back to Lecture 3.

LECTURE 7 · WRAP-UP

The Hyperparameter Map — and What Comes Next

MAIN HYPERPARAMETER MAP

HYPERPARAMETERS

OPTIMIZATION
- Learning Rate
- Optimizer
- Warmup
- LR Scheduler

DATA / MEMORY
- Batch Size
- Sequence Length
- Grad Accumulation

REGULARIZATION
- Weight Decay
- Dropout
- Grad Clipping

PEFT / LoRA · Rank · Alpha · LoRA Dropout · Target Modules

INFERENCE / GENERATION · Temperature · Top-k · Top-p · Max New Tokens

FINAL MENTAL MODEL

Hyperparameters → Control → **HOW the model trains** → affects

Quality · Stability · Memory · Training Time · Generalization · Cost

no single configuration works best everywhere

FIVE TO REMEMBER

1. **Learning Rate** — how large are optimization steps?
2. **Batch Size** — how many examples contribute to an update?
3. **Epochs** — how many times do we revisit the dataset?
4. **Sequence Length** — how much text can each example contain?
5. **Regularization / PEFT configuration** — how much capacity and constraint do we give adaptation?

"If there are so many interacting hyperparameters, how do we find the best combination without trying every possible value manually?"

→ LECTURE 8 — Hyperparameter Optimization + Complete Production Case Study

`Manual Tuning` → `Grid Search` → `Random Search` → `Bayesian / Intelligent Search` → `Experiment Tracking` → `Select Best Configuration` → `Complete Production Pipeline`

"Lecture 7 tells us what each knob controls. Lecture 8 will show us how to tune those knobs systematically."